The Open Source Audit

A Capstone Project for OSS NGMC Course

STUDENT NAME	REGISTERATION NUMBER
Divyani Singh	24MIP10185

SLOT	DATE OF SUBMISSION
A24	20/03/2026

INTRODUCTION

When we think about the software that runs the world around us, we rarely stop to ask who built it, why they shared it, and what that means for the rest of us. Most of the tools that developers use daily, the servers that host websites, and the databases that store our information were not built behind closed doors by corporations looking to profit. They were built openly, shared freely, and improved by communities of people who believed that knowledge should not have a price tag.

This project is my attempt to understand that world more deeply.

I chose MySQL as the subject of this audit because it represents something genuinely interesting in the open source landscape. It is not just a database — it is a story about a small Swedish company that built something powerful, shared it with the world, got acquired by one of the largest technology corporations on the planet, and then watched its own creators walk away and start over rather than see their work locked down. That story involves licensing battles, community loyalty, corporate strategy, and a fundamental question about who really owns open source software once it becomes valuable.

Through this project I have tried to go beyond the surface . I read the actual GPL v2 license text rather than just looking up a summary . I installed MySQL on a Linux system and traced where it lives on the filesystem, what permissions it runs under, and how it behaves as a service. I wrote five shell scripts that connect the philosophy of open source to the practical reality of working on Linux. And I tried to think honestly about the harder questions — whether open source is always better, whether it is ethical for corporations to profit from community labor, and what it means to build something and give it away.

PHILOSOPHY

To understand why MySQL exists, you need to go back to the early 1990s when the internet was just starting to become something everyday people could actually use. Websites were being created, businesses were moving online, and developers were facing a problem that seems almost silly today—there was no affordable, fast, and reliable way to store and retrieve data for a web application.

The big database systems of that time, like Oracle and IBM DB2, were powerful but also too expensive for small developers, students, and startups. If you were building something on the early web and needed a database, you either paid a fortune for a commercial license or struggled with tools that weren't really designed for the job.

That gap is what Michael Widenius, known as Monty, and his colleague David Axmark decided to fill. Monty, a Swedish programmer, had been working with databases since the 1970s. By the mid-1990s, he had a clear view of what developers really needed—something fast, stable, capable of handling the read-heavy workloads that websites create, and something that wouldn't break the bank. In 1995, working from Sweden under a company called MySQL AB, they released the first version of MySQL to the world.

The name itself carries a bit of history. The "My" in MySQL comes from Monty's daughter, My. The "SQL" stands for Structured Query Language, the standard language used for interacting with relational databases. It's a very human detail in what could seem like a purely technical tale—a father named his biggest creation after his child.

What MySQL offered was simple but genuinely useful. It was fast, especially for reading data, which is what most websites do most of the time. It was free to download and use. It also ran smoothly on Linux, which was quickly becoming the go-to operating system for web servers. MySQL, Linux, Apache, and PHP formed the four pillars of what developers called the LAMP stack—the foundation on which a generation of the internet was built. Wikipedia, early Facebook, early Twitter, WordPress—all of them ran on MySQL.

For a while, the story was a straightforward open-source success. MySQL AB grew, the community around MySQL expanded, and the software became one of the most widely used databases worldwide. But then things turned complicated in a way that highlights the tension between open-source ideals and the reality of business.

In 2008, Sun Microsystems bought MySQL AB for about one billion dollars. That figure alone suggests that free software had become incredibly valuable. The open-source community watched with concern. Then in 2010, Oracle acquired Sun Microsystems, and suddenly, MySQL was owned by the very company whose expensive proprietary database it had been created to challenge. Oracle owned both the underdog and the champion.

Monty Widenius didn't wait to see what Oracle would do. Before the acquisition was even final, he started working on a fork of MySQL called MariaDB, named after his younger daughter Maria. He took the original code, the core values, and much of the original development team and began anew, ensuring that the project would stay genuinely open. Today, MariaDB is the default MySQL replacement in many major Linux distributions, including Fedora and Debian.

The MySQL story isn't just a technical history. It shows what happens when something created for free becomes valuable enough for companies to fight over. It raises tough questions about ownership, community trust, and whether a company can ever truly give something away once it realizes it's worth keeping. These questions make MySQL a fascinating topic for any project about open-source software.

License & Ethics

Part A2 — The License Analysis

When I first checked out the MySQL license, I figured it'd be pretty simple. Turns out, it's anything but. It's actually pretty weird—MySQL exists under two totally different licenses at the same time, and which one applies to you depends on who you are and what you want to do.

For community and open source users, MySQL is licensed under the GNU General Public License version 2, or GPL v2 for short. But if you're a business and want to avoid the obligations baked into the GPL, you can buy a commercial license. This dual-licensing approach doesn't just happen by accident. It's all about what the GPL says and why it matters.

- What are the four freedoms of free software? Does your software's license grant all four?

Richard Stallman and the Free Software Foundation sum up "free software" with four freedoms. First, you get to run the program for any purpose. Second, you can look under the hood, figure out how it works, and tweak it—all because you're allowed to see the source code. Third, you're free to share copies with others. Fourth, you can distribute your own tweaked versions.

With the GPL v2, MySQL gives you all of that. Anyone can download it, poke through the code, change it, and send it around. That's as open as software gets. But there's a catch, especially for companies: if you share software that includes GPL code, you have to release your own work under the GPL, too. That's the "copyleft" rule—it's what really separates GPL from more relaxed licenses.

- Can a Company Take MySQL, Modify It, and Sell It Without Sharing?

The short answer? Not happening, at least not under GPL v2. If a company grabs MySQL, messes with it, and releases it as part of their product, they have to publish their changes under the GPL. They can't keep improvements to themselves and sell them as closed software.

This is basically why MySQL AB started offering a commercial license. If a company wants to tuck MySQL into a proprietary product—something they sell but don't share code for—they can pay for a license that skips the copyleft rule. That's how MySQL AB made money while giving the community free access. Oracle still does the same thing.

- GPL vs MIT — What Is the Difference?

The MIT license is "permissive"—take the code, change it, use it however, even in proprietary stuff, and there's no obligation to share your changes. You can build closed commercial products and owe nothing to the original community.

GPL, on the other hand, is all about reciprocity. If you use and share GPL code, your code has to be open, too. It's meant to keep software and its improvements free for everyone, always.

If I were picking a license for something I built, honestly, it comes down to my goals. If I care most about getting my project used everywhere—even by companies who won't give back—I'd go with MIT. If I want a real community vibe, and I want every improvement to stay open, GPL is the way to go. MySQL's dual licensing is actually a smart compromise—it builds a strong community thanks to the GPL, and makes money with the commercial option. It's kind of a straight-up approach, even if it feels a bit strange sometimes.

- Free as in Free Beer vs Free as in Freedom

This is one of those distinctions that's a big deal in open source, and MySQL is a textbook example. "Free as in free beer" means you don't have to pay—it's just free to grab. "Free as in freedom" means you're allowed to use, study, change, and share it.

For the community, MySQL under GPL is both: it doesn't cost anything, and you get real freedom. For companies who need the commercial license, things shift: MySQL is still "free as in freedom" for the community, but not "free as in free beer" for them—they have to pay. That freedom comes with strings attached when you want to make money from it, and honestly, that's exactly how the GPL is supposed to work.

Part A3 — Ethical Reflection

- Should All Software Be Open Source?

There's a strong argument for making everything open. I mean, you could say knowledge built on public money and maintained by public resources should belong to everyone. Open source lets anyone fix security issues, keeps software alive even if a company goes bust, and levels the playing field—anyone anywhere can use the same tools.

But writing good software isn't cheap. Skilled people need to pay their bills. Most open source projects survive because a handful of committed folks do the work and the rest of the world just uses it. Burnout is common. Sometimes massive libraries—like Log4j during the Log4Shell crisis—are kept running by volunteers in their spare time. Forcing all software to be open source without fixing the money problem would probably just make things worse.

Honestly, I don't think "Should all software be open source?" is even the right question. The real issue is whether the people behind open source are treated fairly, especially when huge companies cash in on their work.

- Is It Ethical for Companies to Profit From Free Community Labor?

This is where things get tricky—and MySQL is a perfect case study. Red Hat turned Linux, a volunteer-run project, into a billion-dollar company. Google relies on open source tools like Linux, Apache, MySQL, Python, and plenty more. Meta put React into the world, but

let's be real, their engineers were paid to solve Meta's own problems—so yes, they shared, but they benefitted most.

I don't think it's fundamentally wrong for companies to build on open source and make money—the licenses allow it, and the original developers chose those rules on purpose. What feels sketchy to me is when businesses take massive value from open source while refusing to contribute back in any meaningful way—whether that's code, money, even just acknowledging where the value came from.

Oracle's an interesting example. They keep releasing community versions of MySQL, but the open source crowd never really got over the acquisition, and the rise of MariaDB shows how hard it is to regain trust once it's gone. The original creators walked away.

- Standing on the Shoulders of Giants means the context of software? Does open source enables or undermine original innovation?

Newton's famous phrase—"standing on the shoulders of giants"—fits software better than he could've imagined. Every app I build relies on operating systems, languages, and databases created by other people. The whole modern tech world stands on open source foundations, put there by folks who chose sharing over selling.

I think open source is, if anything, a huge driver of original innovation. There's proof everywhere. The LAMP stack made a generation of web startups possible. Linux running on Android put a free OS in billions of hands. MySQL made it possible for a student to build projects that could handle millions of users.

What open source asks for in return isn't money. It's about being upfront about what you're building on, and maybe pitching in—whether that's code, docs, bug reports, or just paying the maintainers whose work powers your stuff. That's fair. Whether the tech industry actually lives up to that deal? I'm not so sure. Honestly, there's no easy answer.

LINUX FOOTPRINT

Installing MySQL on Linux is pretty simple, especially if you're running a Debian-based distro like Ubuntu. You just use the apt package manager—no need to fuss around with compiling anything yourself. When you run sudo apt install mysql-server, it grabs the package, sorts out all the dependencies for you, and sets up the service so it's ready to go. That's the beauty of popular open source software—it gets maintained and packaged by the broader community, not just the original developers.

After you install MySQL, the program spreads itself across a few key Linux directories. Each one serves its own purpose, basically following the usual Linux filesystem layout.

First up, configuration files are in /etc/mysql. MySQL checks this folder for its settings whenever it starts. Here you'll find things like which port it uses, where it stores data, how much memory it grabs, and so on. The main config file is mysqld.cnf.

The actual database files—the real, raw data MySQL holds—are tucked away in /var/lib/mysql. That's the one directory you absolutely care about if you're backing stuff up or need to recover data.

Log files go to /var/log/mysql. MySQL records errors, warnings, and other activity here, so it's your first stop when things break or act weird.

If you ever type mysql into the command line, the executable you run lives in /usr/bin/mysql. That's pretty standard for user-facing programs in Linux.

One thing that stood out the first time I checked running processes (ps aux | grep mysql): MySQL doesn't run as root. It runs under its own system user called mysql, part of a group named mysql. Security-wise, this is huge. If something goes wrong and MySQL is compromised, the attacker's stuck with limited access. The mysql user can't mess with system files, change other users' stuff, or grab more privileges easily. Running services as low-privilege users is a basic but crucial part of Linux's security approach.

MySQL sets itself up as a systemd service, so you manage it using the usual systemctl commands. If you want to see if it's running, use sudo systemctl status mysql. Start it with sudo systemctl start mysql. Stop it with sudo systemctl stop mysql. Restart it after changing configs with sudo systemctl restart mysql. And if you want it to start automatically when the system boots, sudo systemctl enable mysql takes care of that. Since it plugs right into systemd, MySQL acts like a proper Linux application—it starts with

the system, logs stuff through standard channels, and you can monitor or control it just like any other service.

When it comes to updates, this is where open source really shines. Because the Ubuntu community handles MySQL packages, security updates come through the same apt system as everything else. Just run sudo apt update followed by sudo apt upgrade, and you'll grab the latest patches, including any for MySQL. Since MySQL is open source, anyone in the community can spot a problem, fix the code, and push out an update—usually faster than proprietary vendors who have to slog through endless internal reviews before they even admit there's a problem.

Installation method

```
divvi@rajeev:~$ sudo apt update
sudo apt install mysql-server -y
[sudo] password for divvi:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1839 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [338 kB]
Fetched 2439 kB in 4s (576 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mysql-server is already the newest version (8.0.45-0ubuntu0.24.04.1).
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
divvi@rajeev:~$ mysql --version
mysql  Ver 8.0.45-0ubuntu0.24.04.1 for Linux on x86_64 ((Ubuntu))
divvi@rajeev:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
     Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-19 06:29:37 UTC; 2h 53min ago
    Process: 170 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 234 (mysqld)
     Status: "Server is operational"
      Tasks: 37 (limit: 9335)
     Memory: 381.0M (peak: 438.2M)
        CPU: 1min 2.530s
     CGroup: /system.slice/mysql.service
             └─234 /usr/sbin/mysqld

Mar 19 06:29:33 rajeev systemd[1]: Starting mysql.service - MySQL Community Server...
Mar 19 06:29:37 rajeev systemd[1]: Started mysql.service - MySQL Community Server.
divvi@rajeev:~$
```

Key directories

```
ls /var/log/mysql
ls /usr/bin/mysql
conf.d  debian-start  debian.cnf  my.cnf  my.cnf.fallback  mysql.cnf  mysql.conf.d
error.log
/usr/bin/mysql
divvi@rajeev:~$ ps aux | grep mysql
mysql        234  1.0  4.9 2376228 393044 ?      Ssl  07:47   1:05 /usr/sbin/mysqld
divvi       1383  0.0  0.0   4088  1920 pts/0    S+   09:27   0:00 grep --color=auto mysql
divvi@rajeev:~$ ls -ld /etc/mysql
ls -ld /var/log/mysql
ls -ld /var/lib/mysql
drwxr-xr-x 4 root root 4096 Mar 19 06:07 /etc/mysql
drwxr-x--- 2 mysql adm 4096 Mar 19 06:07 /var/log/mysql
drwx------ 7 mysql mysql 4096 Mar 19 06:29 /var/lib/mysql
divvi@rajeev:~$
```

Service management

```
divvi@rajeev:~$ sudo systemctl start mysql
sudo systemctl stop mysql
sudo systemctl restart mysql
divvi@rajeev:~$ sudo apt list --upgradable | grep mysql

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

divvi@rajeev:~$ sudo mysql
```

SCRIPT 1 and 2

```
divvi@rajeev:~$ #!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Reg No: [Your Reg No]
# Course: Open Source Software | Software chosen: MySQL
# Purpose: Displays system information as a welcome screen

# --- Variables ---
STUDENT_NAME="[Your Name]"        # Store student name
SOFTWARE_CHOICE="MySQL"            # Store chosen software

# --- Capture system information using command substitution ---
KERNEL=$(uname -r)                 # Get kernel version
USER_NAME=$(whoami)                # Get current logged in user
UPTIME=$(uptime -p)               # Get system uptime in readable format
HOME_DIR=$HOME                    # Get home directory of current user

# Get Linux distribution name from os-release file
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')

# Get current date and time in readable format
CURRENT_DATE=$(date '+%d %B %Y, %H:%M:%S')

# --- Display formatted output ---
echo "==========================================="
echo "   Open Source Audit - $STUDENT_NAME"
echo "   Chosen Software : $SOFTWARE_CHOICE"
echo "==========================================="
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $CURRENT_DATE"
echo "-------------------------------------------"
echo "Note: The Linux OS on this system is covered"
echo "under the GNU General Public License v2 (GPLv2)"
echo "==========================================="
===========================================
   Open Source Audit - [Your Name]
   Chosen Software : MySQL
===========================================
Distribution : Ubuntu 24.04.4 LTS
Kernel       : 6.6.87.2-microsoft-standard-WSL2
User         : divvi
Home Dir     : /home/divvi
Uptime       : up 2 hours, 23 minutes
Date/Time    : 19 March 2026, 09:41:54
-------------------------------------------
Note: The Linux OS on this system is covered
under the GNU General Public License v2 (GPLv2)
===========================================
```

Script 1 works like a welcome screen—it kicks things off by introducing the Linux system and tying it to the open source audit you're running. Right away, it sets up two variables:

STUDENT_NAME and SOFTWARE_CHOICE. These just grab simple ID details, which show up in the header of the output.

The heart of the script is command substitution—it uses $(...) to grab information from the system without printing it straight out. You see this trick four times in the script. uname -r gives you the kernel version, whoami tells you which user is logged in, uptime -p explains how long the system's been running in a friendly way, and date (with a formatting string) prints a clean timestamp.

To get the distribution name, the script digs into /etc/os-release. This file is always there on modern Linux systems and holds basic details about the OS. A pipe connects cat, grep, cut, and tr, so you end up with just the PRETTY_NAME value, minus any extra quotes.

For output, echo takes care of formatting—lines, spaces, separators, all to make things easy to read. At the end, there's a nod to the GPL v2 license, bringing everything back to the bigger idea here: the OS running this audit is open source, under the same license type as MySQL.

So, what does the script show you? Handling variables, swapping command output with $(), shaping strings with echo, chaining multiple commands using pipes, and pulling info from system files.

```
divvi@rajeev:~$ #!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Reg No: [Your Reg No]
# Course: Open Source Software | Software chosen: MySQL
# Purpose: Checks if MySQL is installed and displays package info

# --- Define package to inspect ---
PACKAGE="mysql-server"            # Package name to check

echo "==========================================="
echo " FOSS Package Inspector"
echo " Checking package: $PACKAGE"
echo "==========================================="

# --- Check if package is installed using if-then-else ---
if dpkg -l | grep -iq "$PACKAGE"; then
    # Package found — display installation details
    echo "STATUS  : $PACKAGE is INSTALLED on this system."
    echo "-------------------------------------------"
    echo "Package Details:"

    # Use dpkg to get version and description, filter with grep
    dpkg -l | grep -i "$PACKAGE" | awk '{print "Version : " $3}'
    echo "-------------------------------------------"

    # Show where the binary is located
    echo "Binary Location:"
    which mysql

    # Show license info
    echo "-------------------------------------------"
    echo "License : GPL v2 / Commercial (Dual License)"
else
    # Package not found
    echo "STATUS  : $PACKAGE is NOT installed."
    echo "To install run: sudo apt install mysql-server"
fi

echo "==========================================="

# --- Case statement: philosophy note per package ---
echo "Open Source Philosophy Note:"
echo "-------------------------------------------"

case $PACKAGE in
    mysql-server|mysql)
        echo "MySQL: Dual-licensed under GPL v2 and commercial."
        echo "The database that powered a generation of the web."
        echo "Born free, acquired by Oracle, forked into MariaDB."
        ;;
    apache2|httpd)
```

```
    apache2|httpd)
echo "==========================================="g deeply."led him."
==========================================
 FOSS Package Inspector
 Checking package: mysql-server
==========================================
STATUS  : mysql-server is INSTALLED on this system.
------------------------------------------
Package Details:
Version : 8.0.45-0ubuntu0.24.04.1
Version : 8.0.45-0ubuntu0.24.04.1
Version : 8.0.45-0ubuntu0.24.04.1
------------------------------------------
Binary Location:
/usr/bin/mysql
------------------------------------------
License : GPL v2 / Commercial (Dual License)
==========================================
Open Source Philosophy Note:
------------------------------------------
MySQL: Dual-licensed under GPL v2 and commercial.
The database that powered a generation of the web.
Born free, acquired by Oracle, forked into MariaDB.
==========================================
divvi@rajeev:~$
```

Script 2 checks if MySQL is installed on your system, but it's not just a technical exercise—it ties back to the open source philosophy behind the project. The script actually shows off a bunch of useful shell scripting tricks working together.

Right from the start, it puts the package name in a variable called PACKAGE. That's smart because you can easily switch out MySQL for something else; just change that one line at the top and you're good, no need to touch anything else.

The main part uses an if-then-else block. It runs dpkg -l to list installed packages on a Debian-based system, then pipes that to grep -iq to search for the package name quietly and without worrying about case. If the package pops up, the script gives you detailed info; otherwise, it prints how to install it. This approach—always handling both the "it's there" and "it's not" situations—keeps things solid and reliable.

When MySQL is found, dpkg -l and awk team up again to pull out and show you the version number cleanly. The which command tells you exactly where the MySQL binary lives on your machine. These bits actually link to Part B of the report, about directory structure and installation locations.

Toward the end, there's a case statement—a classic in bash. It runs through different options based on the PACKAGE value and prints a short philosophy note for whatever package you're inspecting. This part covers MySQL, Apache, VLC, Firefox, Git, and Python, plus a catch-all for anything else.

SCRIPTS 3 AND 4

```
divvi@rajeev: $ nano script3.sh
divvi@rajeev: $ #!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: [Your Name] | Reg No: [Your Reg No]
# Course: Open Source Software | Software chosen: MySQL
# Purpose: Loops through key system directories and reports
#          permissions, ownership and disk usage for each

# --- List of directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "============================================"
echo " Directory Audit Report"
echo " Generated: $(date '+%d %B %Y %H:%M:%S')"
echo "============================================"

# --- For loop through each directory ---
for DIR in "${DIRS[@]}"; do

    # Check if directory exists before processing
    if [ -d "$DIR" ]; then

        # Extract permissions, owner and group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')

        # Get human readable size using du
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "Directory  : $DIR"
        echo "Permission : $PERMS"
        echo "Owner      : $OWNER"
        echo "Group      : $GROUP"
        echo "Size       : $SIZE"
        echo "--------------------------------------------"
    else
        # Directory does not exist on this system
        echo "$DIR does not exist on this system."
        echo "--------------------------------------------"
    fi
done

# --- MySQL specific config directory check ---
echo ""
echo "MySQL Specific Directory Check:"
echo "============================================"

# List of MySQL related directories to check
MYSQL_DIRS=("/etc/mysql" "/var/log/mysql" "/var/lib/mysql")
```

Script 3 audits essential Linux directories, looking at their permissions, owners, groups, and disk usage. It doesn't just stick to the basics—it also checks specific directories used by MySQL, like where its config files, logs, and database data live. That way, the script ties the general system check to what matters for MySQL.

Right at the start, there's an array called DIRS. It's a list of five big system folders. In bash, arrays are just parentheses with the values spaced out inside—and you grab all the elements at once using ${DIRS[@]}.

The heart of the script is a for loop. It goes through each directory, but before doing anything, it checks if the directory exists using the -d flag. That way, there's no risk of errors if one of the folders is missing.

When a directory is found, the script runs ls -ld. That gives a detailed summary about the directory itself, not its contents. Then it pipes the output to awk, which picks out just the

info you care about—permissions, owner, and group. For disk usage, it uses du -sh, so you get a readable size, and cut strips it down to just the value.

The second part does the same thing, but zooms in on MySQL directories— /etc/mysql (configs), /var/log/mysql (logs), and /var/lib/mysql (data). Checking these matters for security, especially the data directory. For example, /var/lib/mysql usually has drwx------ permissions, meaning only the mysql user can access it. That locks down your database files.

Overall, the script uses a bunch of shell scripting basics: arrays, for loops, if checks, ls with extra flags, awk for picking out info, du for space, cut for trimming output, and command substitution to drop the date in the header.

```
ivvi@rajeev: $ #!/bin/bash
 Script 4: Log File Analyzer
 Author: [Your Name] | Reg No: [Your Reg No]
 Course: Open Source Software | Software chosen: MySQL
 Purpose: Reads a log file line by line, counts keyword
          matches and prints a summary with last 5 matches
 Usage: ./script4.sh /var/log/syslog error

 --- Accept command line arguments ---
OGFILE=$1                          # First argument is log file path
EYWORD=${2:-"error"}               # Second argument is keyword, default is error

 --- Initialise counter variable ---
OUNT=0

cho "============================================"
cho " Log File Analyzer"
cho "============================================"
cho "Log File : $LOGFILE"
cho "Keyword  : $KEYWORD"
cho "--------------------------------------------"

 --- Validate that log file exists ---
f [ ! -f "$LOGFILE" ]; then
   echo "Error: File '$LOGFILE' not found."
   echo "Usage: ./script4.sh <logfile> <keyword>"
   exit 1
i

 --- Do-while style retry if file is empty ---
hile [ ! -s "$LOGFILE" ]; do
   echo "Warning: $LOGFILE is empty."
   read -p "Enter a different log file path: " LOGFILE

   # Check if the new file exists
   if [ ! -f "$LOGFILE" ]; then
       echo "File not found. Exiting."
       exit 1
   fi
one

 --- Read file line by line using while read loop ---
hile IFS= read -r LINE; do

   # Check each line for keyword match using if-then
   if echo "$LINE" | grep -iq "$KEYWORD"; then
       COUNT=$((COUNT + 1))        # Increment counter
   fi
```

```
done < "$LOGFILE"                    # Redirect file into while loop

echo "==============================================="
==============================================
 Log File Analyzer
==============================================
Log File :
Keyword  : error
-----------------------------------------------
Error: File '' not found.
Usage: ./script4.sh <logfile> <keyword>
logout

[process exited with code 1 (0x00000001)]
You can now close this terminal with Ctrl+D, or press Enter to restart.
```

Script 4's all about digging through a log file, counting up the lines that mention a certain keyword, and then showing you the most recent ones. Log analysis is one of those things people do constantly with shell scripts, and this script pulls together a bunch of classic bash tricks.

Here's how it works: You hand it two arguments on the command line—the first is the path to your log file, which it grabs into LOGFILE as $1. The second is the keyword you want to search for. It grabs that into KEYWORD using ${2:-"error"}, which just means, "If you don't give me a second argument, I'll use 'error' by default." Handy, because you can skip typing out common keywords if you want.

There's also a retry loop built in. If the file turns out to be empty (checked with the -s flag), it prompts you to enter a new path and tries again. That way, you don't end up getting a zero-match result from an empty file—because that's just confusing and pointless.

It uses .The heart of the script is a while-read loop that goes through the log file line by line Inside .r flag so it doesn't mess up lines with weird characters or whitespace- IFS= and the iq (so it's case-insensitive and doesn't print - each line gets checked with grep ,that loop it bumps up the COUNT variable. ,When there's a hit .(just tells if there's a match ,anything

After it's done reading, it prints out the total number of matches. Then, it uses grep piped into tail -5 to show you the last five matching lines—usually, those recent events are what you actually want to see.

There's a lot of shell scripting concepts packed in here: command line arguments, default variables, if-then checks, exit codes, while-read loops, handling IFS, counting with arithmetic, using grep with flags, grabbing the last few lines with tail, and input redirection with <. It's a solid example of how all these pieces work together.

ECOSYSTEM

No software does everything solo, and MySQL really shows just how tangled and connected open source projects get. When you pick MySQL for your Linux box, you're not just grabbing one program. You're tapping into decades of other projects, all stitched together. It's kind of mind-blowing if you think about it.

Since MySQL's written mostly in C and C++, it counts on GCC—the GNU Compiler Collection—to even get built. GCC isn't some side project, either. It's a cornerstone of the open source community, maintained by the Free Software Foundation. Without GCC, honestly, MySQL as we know it wouldn't even exist.

Then you've got glibc—the GNU C Library—which takes care of the basic stuff every Linux program needs: memory, files, all that. MySQL leans on OpenSSL for secure connections, which is one of those projects that keeps the whole internet running safely. And for actually building and compiling MySQL, there's CMake, another open source tool. Even the documentation doesn't escape—the team uses open source tools for that, too. Really, every layer is built by people who decided to share their work with everyone.

The story of MySQL's impact is honestly pretty huge. It didn't just solve database problems; it completely changed web development for people who couldn't afford big, expensive licenses. Suddenly, anyone could build a database-backed website.

Then there's MariaDB, which is basically MySQL's twin—but with a twist. It was started by MySQL's original creators after Oracle bought MySQL. Now MariaDB's the default database on some major Linux distros (like Fedora and Debian), and the big thing is: it promises to stay open source. Because MySQL was licensed under the GPL, the community had the legal right to fork it and keep working, even when big companies got involved. That's proof that open source licensing actually matters.

Percona Server is another fork, laser-focused on speed and reliability. Percona also built XtraBackup, which tons of admins use when they need backups that won't slow things down.

The effects go way beyond direct forks. MySQL kicked off a whole conversation about open source databases. That helped push projects like PostgreSQL and even new types of

databases like MongoDB and Redis to become “open source first.”
And then—WordPress. The most common website platform anywhere, and it’s built on MySQL. You see the same with Drupal, Joomla, and other content systems. Web hosting companies built their entire business models around the LAMP stack, with MySQL sitting right in the middle.

In my opinion, it’s hard to find another project that’s had such a dramatic ripple effect in how people build things online. MySQL really did open the web to the masses.

LAMP stands for Linux, Apache, MySQL, PHP. Sounds simple, but it’s wildly important in web dev history. This stack gave anyone with a laptop and some time everything they needed to put up a dynamic website—back in the late ’90s, that was a pretty radical idea.

Linux is the foundation, Apache handles web requests, MySQL keeps your data safe and organized, PHP holds it all together. And here’s what made it truly awesome: all four parts were totally free and open. You didn’t need some rich company’s backing—you could go from idea to launch without spending anything on software. People really took advantage of that, and LAMP became the backbone of modern web hosting.

Things have shifted since then—Nginx has often replaced Apache, Python and Node.js are in the mix, MariaDB sometimes stands in for MySQL—but the basic vibe’s the same: a totally open stack that anyone can use. MySQL played a major role in making this possible.

Honestly, seeing the LAMP stack in action for the first time made me realize just how powerful it is to have tools everyone can use. It sparked a lot of creativity.

The MySQL community’s always been lively, but things changed after Oracle swooped in. Officially, there’s the MySQL Forums where developers swap tips, report bugs, and help each other out. They’ve got mailing lists for dev talk and a public bug tracker where anyone can see what’s broken and what’s fixed.

MySQL still shows up big at open source events—like Percona Live and FOSDEM in Brussels. Lots of people still care about it.

But governance got complicated after the Oracle buyout. Before 2010, development had a much more community-driven flavor—first MySQL AB, then Sun Microsystems, with lots of public input. Now, Oracle makes the big decisions: what gets built, which features land, and when new versions launch.

That’s why the MariaDB fork is such a big deal. It’s run by the MariaDB Foundation, a nonprofit, so it doesn’t answer to any single company. Contributors are a mix of folks, organizations, and universities, but no one has the steering wheel. It’s closer to what many

open source fans want—a foundation-run model, kinda like how the Linux Foundation runs the kernel.

COMPARISON AND SCRIPT 5

```
divvi@rajeev:~$ #!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Reg No: [Your Reg No]
# Course: Open Source Software | Software chosen: MySQL
# Purpose: Asks user three questions interactively and
#          generates a personalised open source philosophy
#          statement, then saves it to a text file

# --- alias concept demonstrated via comment ---
# alias mymanifesto='cat manifesto_$(whoami).txt'
# Running the above alias would display the saved manifesto

echo "============================================="
echo " Open Source Manifesto Generator"
echo " Inspired by the MySQL story"
echo "============================================="
echo ""
echo "Answer three questions to generate your manifesto."
echo ""

# --- Read user input interactively ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- Get current date and logged in username ---
DATE=$(date '+%d %B %Y')
AUTHOR=$(whoami)

# --- Define output filename using string concatenation ---
OUTPUT="manifesto_${AUTHOR}.txt"

echo ""
echo "Generating your manifesto..."
echo "---------------------------------------------"

# --- Write manifesto to file using > and >> operators ---
# > creates or overwrites the file
echo "=============================================" > "$OUTPUT"
echo " My Open Source Manifesto" >> "$OUTPUT"
echo " Author : $AUTHOR" >> "$OUTPUT"
echo " Date   : $DATE" >> "$OUTPUT"
echo "=============================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# --- Compose paragraph using string concatenation ---
echo "Every day I rely on $TOOL — a tool I did not pay for," >> "$OUTPUT"
echo "built by people I have never met, who believed that" >> "$OUTPUT"
echo "$FREEDOM is more valuable than profit." >> "$OUTPUT"
```

```
echo "============================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# --- Compose paragraph using string concatenation ---
echo "Every day I rely on $TOOL — a tool I did not pay for," >> "$OUTPUT"
echo "built by people I have never met, who believed that" >> "$OUTPUT"
echo "$FREEDOM is more valuable than profit." >> "$OUTPUT"
echo "============================================" >> "$OUTPUT"UT"T"T"UT"
============================================
 Open Source Manifesto Generator
 Inspired by the MySQL story
============================================

Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: mysql
2. In one word, what does 'freedom' mean to you? privelege
3. Name one thing you would build and share freely: food

Generating your manifesto...
--------------------------------------------
Manifesto saved to manifesto_divvi.txt
--------------------------------------------
============================================
 My Open Source Manifesto
 Author : divvi
 Date   : 19 March 2026
============================================

Every day I rely on mysql — a tool I did not pay for,
built by people I have never met, who believed that
privelege is more valuable than profit.

MySQL taught me that open source is not just a license.
It is a choice about who gets to participate in
technology. When Oracle acquired MySQL, the original
creators walked away and built MariaDB rather than
compromise that choice. That is what commitment to
open source actually looks like in practice.

Someday I will build food and share it freely,
so someone I will never meet can build something
I could never imagine. That is the promise of
open source software.

— divvi
============================================
============================================
```

Script 5 stands out from the rest—it's more personal and creative. Instead of just spitting out system details or crunching through files, it takes those philosophical themes from the project and turns them into something interactive. You answer three questions, and the script mixes your responses into a custom open source philosophy statement. Then it saves it to a file, so it's yours.

The script kicks off with a comment showing how aliases work in bash. Basically, an alias is a shortcut: you can call a custom command name instead of typing out something longer. The comment demonstrates that, after running the script, you could type "mymanifesto" to display your saved statement without using the full cat command. The alias itself isn't actually activated in the script, but the comment walks you through the syntax, showing you understand how it works.

To grab your answers, the script uses the read command with the -p flag. That pops up a prompt, waits for you to type something, and then stores your answer in a variable. There

are three variables: TOOL, FREEDOM, and BUILD. Throughout the manifesto, those are dropped into different places using string concatenation—basically combining the fixed text with your answers inside echo statements.

Saving your statement uses two file redirection operators. The first echo uses “>” to create a fresh file, wiping anything already there. Then, the rest use “>>” to add new lines without overwriting what’s already written. That’s key—if you accidentally use “>” again later, you’d erase everything you’d just recorded.

There’s also a date command with a format string that sticks a human-friendly date in the header of your file. Once everything’s written, the script calls cat to show you your manifesto right away.

Part D — Open Source vs Proprietary Comparison

Comparison Table

Dimension	MySQL (Open Source)	Microsoft SQL Server (Proprietary)
Cost	Free under GPL v2 for community use. Commercial license available for proprietary use	Paid licensing starting from hundreds to thousands of dollars per server per year
Security	Source code is publicly visible — anyone can audit it, find vulnerabilities and contribute fixes	Source code is closed — security depends entirely on Microsoft's internal team
Support	Community forums, Stack Overflow, official documentation, paid support from Oracle available	Official Microsoft support included with license, dedicated enterprise support tiers
Freedom to modify	Full freedom to modify under GPL — MariaDB exists as proof that this works	No freedom to modify — you use what Microsoft ships
Community vs corporate control	Dual situation — Oracle controls MySQL, but MariaDB Foundation provides genuine community governed alternative	Entirely corporate controlled — Microsoft decides everything

Reliability	Extremely battle tested — runs Wikipedia, WordPress, Facebook's early infrastructure	Also extremely reliable — well suited for Windows enterprise environments
Integration	Works on Linux, Windows, Mac — integrates with almost every language and framework	Best integrated with Windows and Microsoft ecosystem
Overall verdict	Better choice for open environments, web applications, cost-sensitive deployments	Better choice for organisations already deep in Microsoft infrastructure

CONCLUSION

When I first dove into this project, I figured I knew what open source was all about. Free software you could download, code anyone could poke through, projects run by communities instead of big-name companies. Turns out, that idea wasn't wrong—it just barely scratched the surface.

Auditing MySQL taught me that open source is way messier and way more interesting than just a tech definition. It's political, and honestly, it's downright human. MySQL's history packs in more drama than you'd ever expect from a database. Think about it—a Swedish programmer names the project after his daughter and gives it away for nothing. Then, a billion-dollar sale. The original creators split off, unwilling to compromise what mattered to them, and the community tags along. Oracle ends up holding the most famous open source database, but the software its creators truly care about goes by a completely different name. Crazy, right? This isn't just a licensing tale—it's a story about trust, values, and what it really means to offer something to the world.

When I sat down to write those five shell scripts for this project, the lesson was different, but just as real. Running commands on a live Linux system, figuring out where MySQL keeps its files, checking which user is in charge, reading log files line by line—that's not some theory cooked up in a textbook. This is the kind of hands-on reality you get when you work with open source for a living. And honestly, every single command happened because someone decided to share their work instead of locking it away. From the terminal, to bash, to tools like grep, awk, and du, and the Linux kernel under all of it—people built these, and they did it so we could use them freely.

This project kicked up some ethical questions I'm not going to shake any time soon. Should corporations make money off the work the community does? Can you trust Oracle to look after MySQL? Is the open source model even sustainable when so much of it rests on volunteers who rarely get paid or even noticed? I don't have easy answers, but now I see why these questions matter.

If there's one thing that's shifted for me in how I see software, it's this: Every line of code I write sits on top of foundations I didn't build, kept alive by people I'll probably never meet, governed by licenses that reflect someone's beliefs about how knowledge should move. You can't ignore that. If you want to call yourself a professional—and be honest about it—

you have to understand where your work fits in.

MySQL gave the web a database anybody could afford. The community made sure MySQL had a future beyond Oracle's control. And this project gave me a new perspective on the software I use every day—a perspective I didn't have until I started.